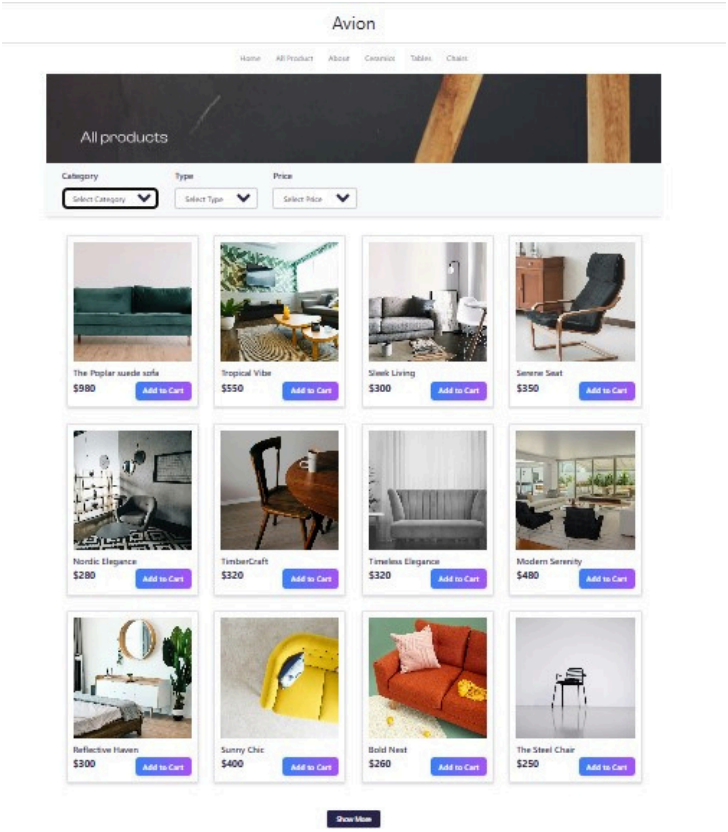
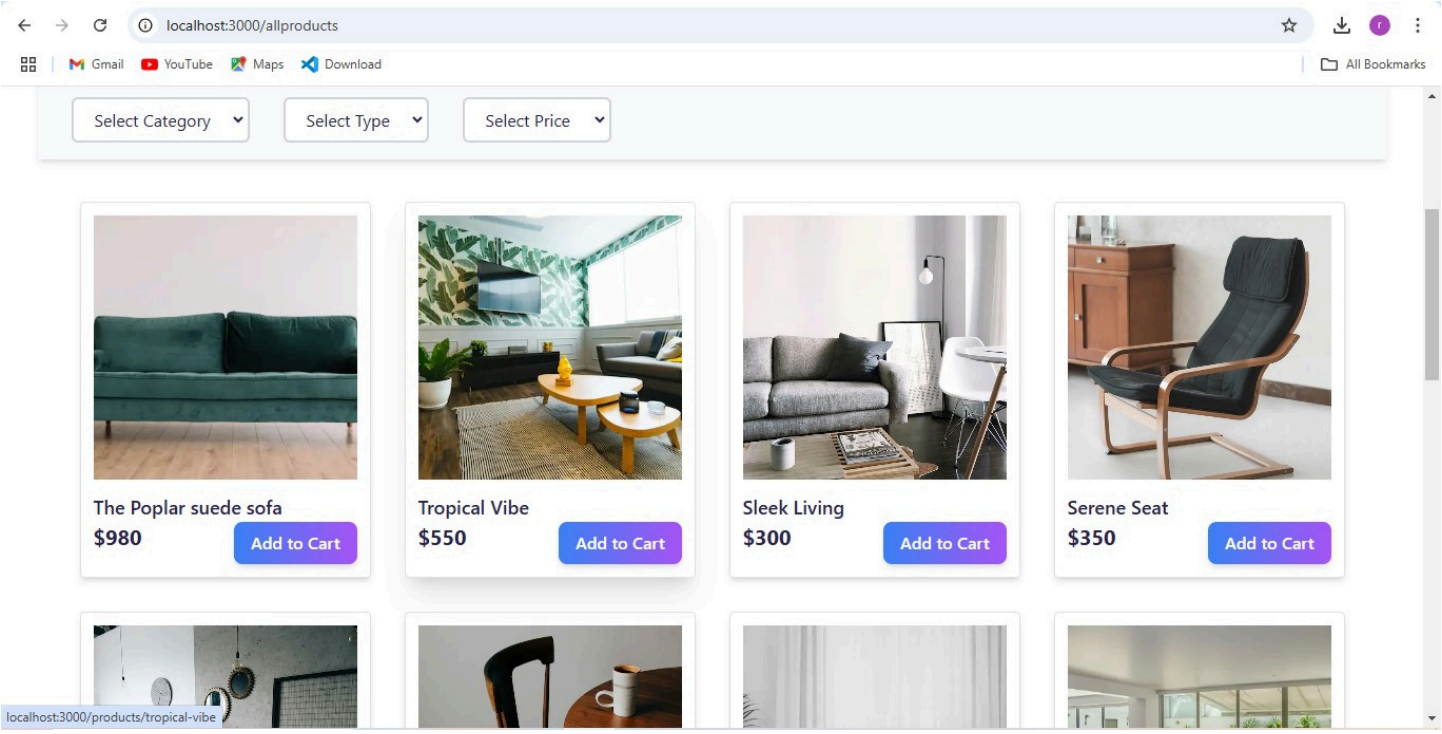
General E-Commerce Marketplace

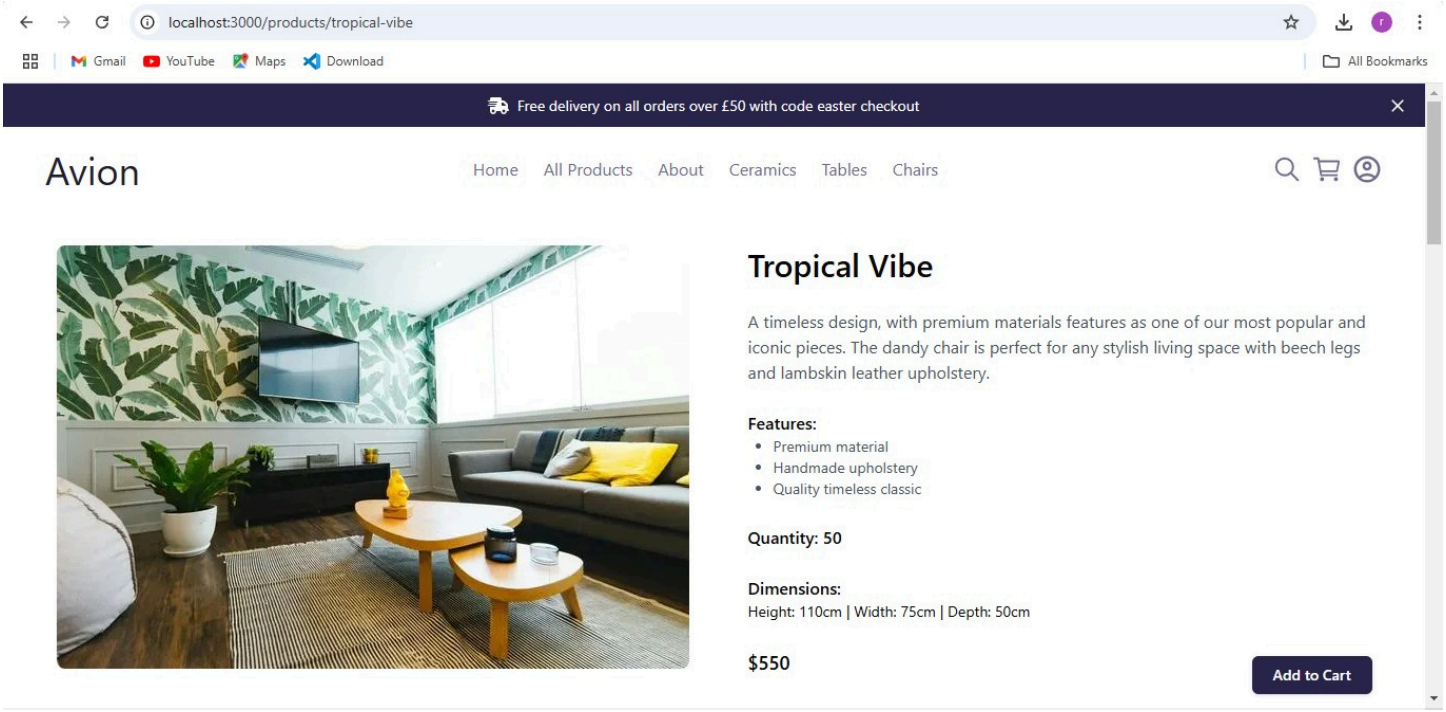
1. Functional Deliverables:

Product listing page screenshot:

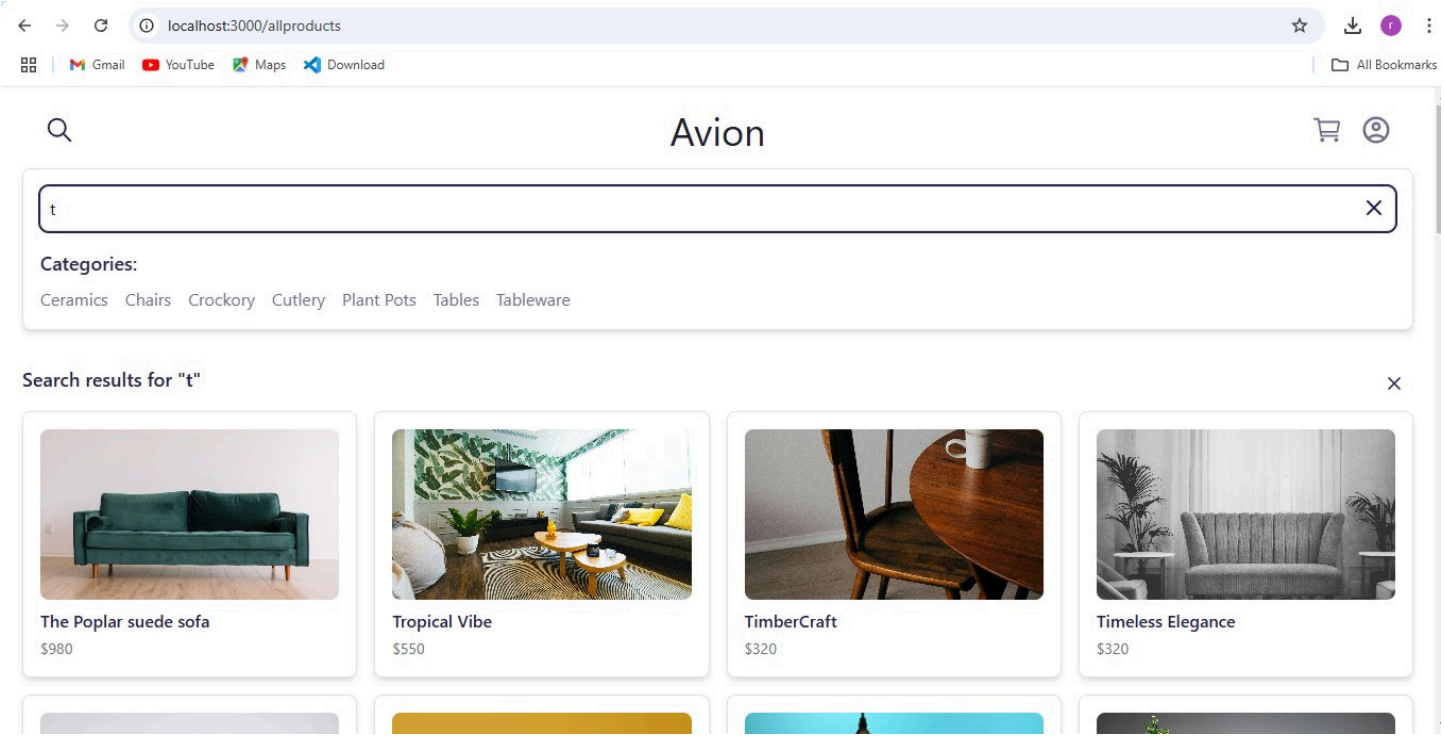


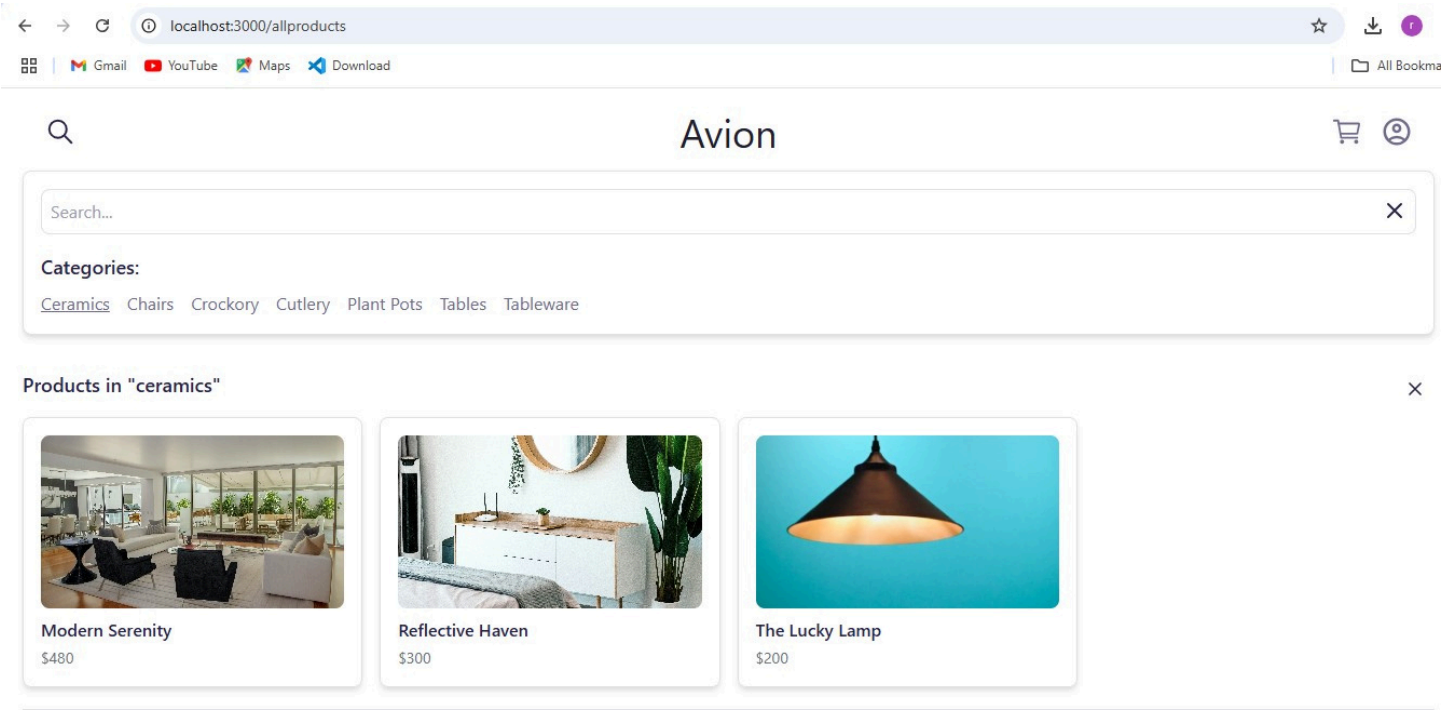
Individual product detail page screenshot:



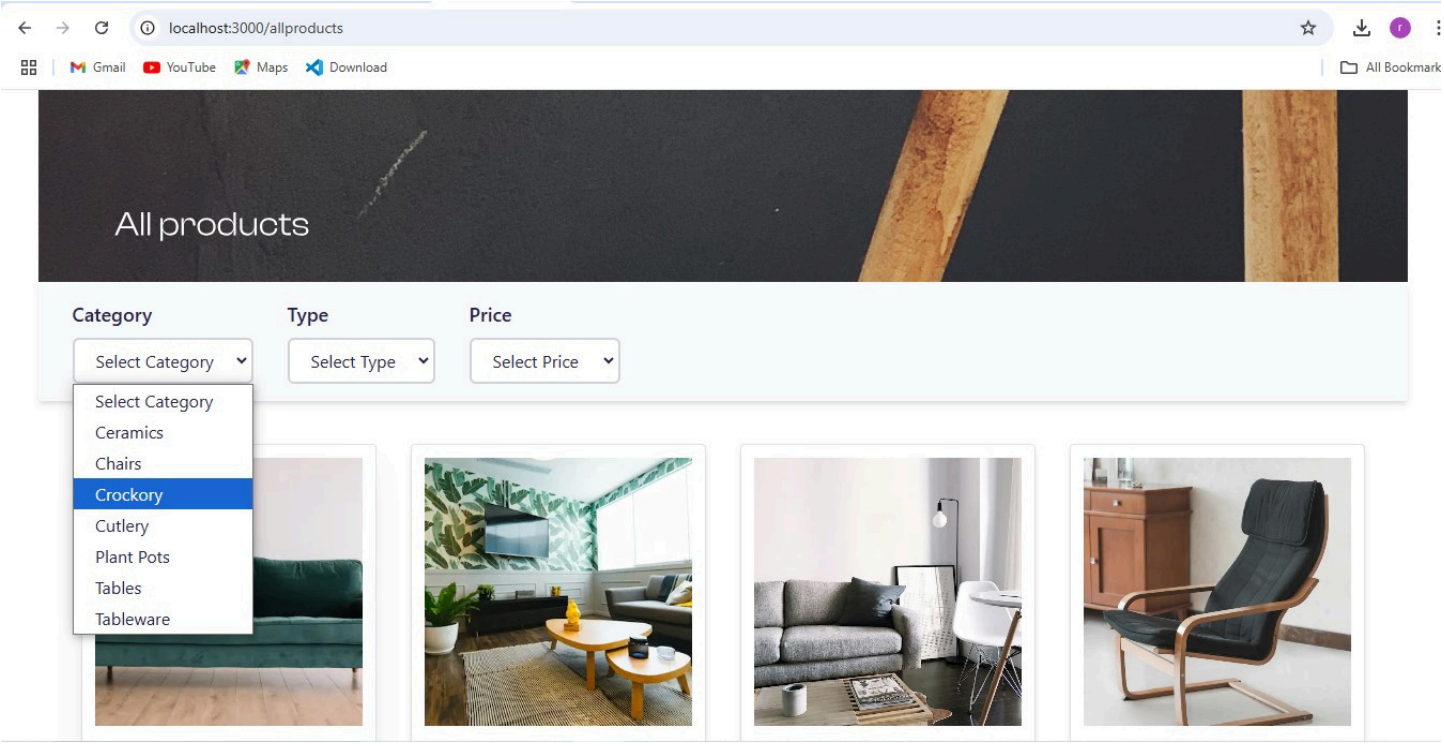


Search bar in action:





Filter in action:



2. Code Deliverables:

AllProducts component code:

```
app > allproducts > page.tsx > IProduct > category
"use client";
import React, { useState, useEffect } from "react";
import Image from "next/image";
import Header from "@/components/Header";
import Footer from "@/components/Footer";
import { client } from "@/sanity/lib/client";
import { urlFor } from "@/sanity/lib/image";
import Link from "next/link";
import { Button } from "@/components/ui/button";

import Swal from "sweetalert2";
import { addToCart } from "../action/page";

interface ICategory {
  name: string;
  slug: { current: string };
}

interface IProduct {
  _id: string;
  name: string;
  slug: { current: string };
  image: { asset: { url: string } };
  price: number;
  category: { slug: { current: string } };
  type: string;
  inventory: number;
}
const AllProducts = () => {
  const [data, setData] = useState<IProduct[]>([]);
  const [categories, setCategories] = useState<ICategory[]>([]);
  const [loading, setLoading] = useState(true);
  const [selectedCategory, setSelectedCategory] = useState<string>("");
  const [selectedType, setSelectedType] = useState<string>("");
  const [selectedPrice, setSelectedPrice] = useState<string>("");
  const [cart, setCart] = useState<IProduct[]>([]);

  const [visibleCount, setVisibleCount] = useState(12);
  const productsPerPage = 12;

  const [showFilters, setShowFilters] = useState(false);
```

pp > allproducts > page.tsx > IProduct > category

```
const AllProducts = () => {

  useEffect(() => {
    const fetchData = async () => {
      try {
        const productQuery = `*[_type == "product"
          && (${selectedCategory ? `category->slug.current == "${selectedCategory}"` : "true"} )
          && (${selectedType ? `type == "${selectedType}"` : "true"} )
          && (${selectedPrice ? `price <= ${selectedPrice}` : "true"})]{
          name, slug, image, price, category, type
        }`;

        const fetchedProducts = await client.fetch(productQuery);
        setData(fetchedProducts);

        const categoryQuery = '*[_type == "category"]{name, slug}';
        const fetchedCategories = await client.fetch(categoryQuery);
        setCategories(fetchedCategories);
      } catch (error) {
        console.error("Error fetching data: ", error);
      } finally {
        setLoading(false);
      }
    };

    fetchData();
  }, [selectedCategory, selectedType, selectedPrice]);

  const handleViewToggle = () => {
    if (visibleCount >= data.length) {
      setVisibleCount(12);
    } else {
      setVisibleCount((prev) => prev + productsPerPage);
    }
  };

  if (loading) {
    return (
      <div className="flex justify-center items-center w-full h-screen">
        <div className="border-8 border-t-8 border-gray-300 border-t-[#2A254B] rounded-full w-16 h-16 animate-spin"></div>
      </div>
    );
  }

  if (data.length === 0) {
    return <p>No products found.</p>;
  }
```

app > allproducts > page.tsx > IProduct > category

```
const AllProducts = () => {

  const handleAddtoCart = (e: React.MouseEvent, product: IProduct) => {
    e.preventDefault();
    Swal.fire({
      position: "top-right",
      icon: "success",
      title: `${product.name} added to cart`,
      showConfirmButton: false,
      timer: 1000,
    });
    addToCart(product); // Add product to cart
  };

  return (
    <div>
      <Header />
      <div className="w-full max-w-screen-xl mx-auto text-[#2A254B]">
        <div>
          <Image
            src="/images/productlisting200.svg"
            alt="Product Listing Banner"
            height={209}
            width={1440}
            className="object-cover w-full"
          />
        </div>

        {/* Mobile Filter Button */}
        <div className="md:hidden flex justify-between items-center py-4 px-8">
          <button
            className="bg-[#F9F9F9] py-2 px-4 text-sm font-semibold rounded-md hover:bg-[#2A254B] hover:text-white transition-colors"
            onClick={() => setShowFilters(!showFilters)}
          >
            {showFilters ? "Hide Filters" : "Show Filters"}
          </button>
        </div>

        {/* Filter section for small screens */}
        {showFilters && (
          <div className="md:hidden w-full px-8 pb-8">
            <div className="flex flex-col gap-6">
              {/* Category Filter UI */}
              <div className="flex flex-col gap-2">
                <label htmlFor="category" className="text-lg font-medium">
                  Category
                </label>
                <select
                  id="category"
```

```
const AllProducts = () => {
                id="category"
                value={selectedCategory}
                onChange={(e) => setSelectedCategory(e.target.value)}
                className="border-2 border-gray-300 rounded-md px-4 py-2 "
              >
                <option value="">Select Category</option>
                {categories.map((category) => (
                  <option
                    key={category.slug.current}
                    value={category.slug.current}
                  >
                    {category.name}
                  </option>
                ))}
              </select>
            </div>

            {/* Type Filter UI */}
            <div className="flex flex-col gap-2">
              <label htmlFor="type" className="text-lg font-medium">
                Type
              </label>
              <select
                id="type"
                value={selectedType}
                onChange={(e) => setSelectedType(e.target.value)}
                className="border-2 border-gray-300 rounded-md px-4 py-2 "
              >
                <option value="">Select Type</option>
                <option value="furniture">Furniture</option>
                <option value="decor">Decor</option>
              </select>
            </div>

            {/* Price Filter UI */}
            <div className="flex flex-col gap-2">
              <label htmlFor="price" className="text-lg font-medium">
                Price
              </label>
              <select
                id="price"
                value={selectedPrice}
                onChange={(e) => setSelectedPrice(e.target.value)}
                className="border-2 border-gray-300 rounded-md px-4 py-2 "
              >
                <option value="">Select Price</option>
                <option value="50">Under $50</option>
```

```
const AllProducts = () => {
                <option value="100">Under $100</option>
                <option value="200">Under $200</option>
              </select>
            </div>
          </div>
        </div>
      )}

      {/* Filter section for large screens */}
      <div className="hidden md:flex gap-8 text-sm sm:text-base px-8 py-4 bg-slate-50 shadow-md">
        {/* Category Filter UI */}
        <div className="flex flex-col gap-2">
          <label htmlFor="category" className="text-lg font-medium">
            Category
          </label>
          <select
            id="category"
            value={selectedCategory}
            onChange={(e) => setSelectedCategory(e.target.value)}
            className="border-2 border-gray-300 rounded-md px-4 py-2 "
          >
            <option value="">Select Category</option>
            {categories.map((category) => (
              <option
                key={category.slug.current}
                value={category.slug.current}
              >
                {category.name}
              </option>
            ))}
          </select>
        </div>

        {/* Type Filter UI */}
        <div className="flex flex-col gap-2">
          <label htmlFor="type" className="text-lg font-medium">
            Type
          </label>
          <select
            id="type"
            value={selectedType}
            onChange={(e) => setSelectedType(e.target.value)}
            className="border-2 border-gray-300 rounded-md px-4 py-2 "
          >
            <option value="">Select Type</option>
            <option value="furniture">Furniture</option>
            <option value="decor">Decor</option>
```

```
const AllProducts = () => {
            <option value="">Select Type</option>
            <option value="furniture">Furniture</option>
            <option value="decor">Decor</option>
          </select>
        </div>

        {/* Price Filter UI */}
        <div className="flex flex-col gap-2">
          <label htmlFor="price" className="text-lg font-medium">
            Price
          </label>
          <select
            id="price"
            value={selectedPrice}
            onChange={(e) => setSelectedPrice(e.target.value)}
            className="border-2 border-gray-300 rounded-md px-4 py-2 "
          >
            <option value="">Select Price</option>
            <option value="50">Under $50</option>
            <option value="100">Under $100</option>
            <option value="200">Under $200</option>
          </select>
        </div>
      </div>

      <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-8  pt-2 p-8 sm:p-10">
        {data.slice(0, visibleCount).map((prod) => (
          <div
            key={prod.slug.current}
            className="flex flex-col border rounded-sm shadow-md hover:shadow-xl transition-all p-3 "
          >
            <Link href={`/products/${prod.slug.current}`} passHref>
              <Image
                src={urlFor(prod.image?.asset).url()}
                alt={prod.name}
                width={400}
                height={400}
                className="w-full h-[250px] object-cover mb-3 "
              />
              <h3 className="font-semibold text-lg ">{prod.name}</h3>
              <div className="flex justify-between">
                <p className="text-xl font-bold ">${prod.price}</p>
                <button
                  className="bg-gradient-to-r from-blue-500 to bg-purple-500 text-white font-semibold py-2 px-4 rounded-lg shadow-md hover:shadow-lg hover:scale-110 transition-transform duration-300 ease-in-c
                  onClick={(e) => handleAddtoCart(e, prod)}
                >
                  Add to Cart
```

```
            <Link href={`/products/${prod.slug.current}`} passHref>
              <Image
                src={urlFor(prod.image?.asset).url()}
                alt={prod.name}
                width={400}
                height={400}
                className="w-full h-[250px] object-cover mb-3 "
              />
              <h3 className="font-semibold text-lg ">{prod.name}</h3>
              <div className="flex justify-between">
                <p className="text-xl font-bold ">${prod.price}</p>
                <button
                  className="bg-gradient-to-r from-blue-500 to bg-purple-500 text-white font-semibold py-2 px-4 rounded-lg shadow-md hover:shadow-lg hover:scale-110 transition-transform duration-300 ease-in-c
                  onClick={(e) => handleAddtoCart(e, prod)}
                >
                  Add to Cart
                </button>
              </div>
            </Link>
          </div>
        ))}
      </div>

      <Button
        onClick={handleViewToggle}
        className="block mx-auto my-4 py-2 px-5 bg-[#2A254B] text-white rounded-sm hover:bg-[#4C3F6B] transition-all"
      >
        {visibleCount >= data.length ? "Show Less" : "Show More"}
      </Button>
    </div>
    <Footer />
  </div>
);
};

export default AllProducts;
```

API integration code for fetching product data:

app > allproducts > page.tsx > IProduct > category

```
const AllProducts = () => {
  const [cart, setCart] = useState<IProduct[]>([]);

  const [visibleCount, setVisibleCount] = useState(12);
  const productsPerPage = 12;

  const [showFilters, setShowFilters] = useState(false);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const productQuery = `*[_type == "product"
          && (${selectedCategory ? `category->slug.current == "${selectedCategory}"` : "true"} )
          && (${selectedType ? `type == "${selectedType}"` : "true"} )
          && (${selectedPrice ? `price <= ${selectedPrice}` : "true"})]{
          name, slug, image, price, category, type
        }`;

        const fetchedProducts = await client.fetch(productQuery);
        setData(fetchedProducts);

        const categoryQuery = '*[_type == "category"]{name, slug}';
        const fetchedCategories = await client.fetch(categoryQuery);
        setCategories(fetchedCategories);
      } catch (error) {
        console.error("Error fetching data: ", error);
      } finally {
        setLoading(false);
      }
    };

    fetchData();
```

Dynamic routing setup code for /product/[slug] page.tsx

HACKATHON-3-TEMPLA...

- src
  - app
    - allproducts
    - cart
    - checkout
    - fonts
    - login
    - product\[id]
    - products\[slug]
      - page.tsx M
    - profile
    - signup
    - studio
    - favicon.ico
    - globals.css
    - layout.tsx M
    - page.tsx M
  - components
  - lib
  - sanity
- .env.local
- .eslintrc.json
- .gitignore
- components.json
- next-env.d.ts

OUTLINE

TIMELINE

IDENTIFY

src > app > products > [slug] > page.tsx > fetchProduct

```
7   import TopBar from "@/components/TopBar";
8   import Brand from "@/components/Brand";
9   import ClubBenefits from "@/components/ClubBenefits";
10  import Footer2 from "@/components/Footer2";
11  import { Button } from "@/components/ui/button";
12
13
14  async function fetchProduct(slug: string) {
15    return await client.fetch(
16      `*[_type == "product" && slug.current == $slug][0]`,
17      { slug }
18    );
19  }
20
21  const ProductPage = async ({ params }: { params: { slug: string } }) => {
22    const product = await fetchProduct(params.slug);
23
24    if (!product) {
25      notFound();
26    }
27
28    return (
29      <div>
30        <TopBar />
31        <ProductDetailHeader />
32
33        <div className="max-w-screen-xl mx-auto px-4 pt-8">
34          <div className="flex flex-col md:flex-row items-center md:items-start">
35            {/* Product Image (Left Side) */}
36            <div className="flex-shrink-0 mb-6 md:mb-0 md:w-1/2">
37              <Image
38                src={urlFor(product.image?.asset).width(600).height(400).url()}
```